

BCC2001

— ORIGIN
'warn'.

monitor

thing. 2 a software
duties. 3 a software
picture for use in
video.

Diego Bertolini

diegobertolini@gmail.com

Aula Passada...

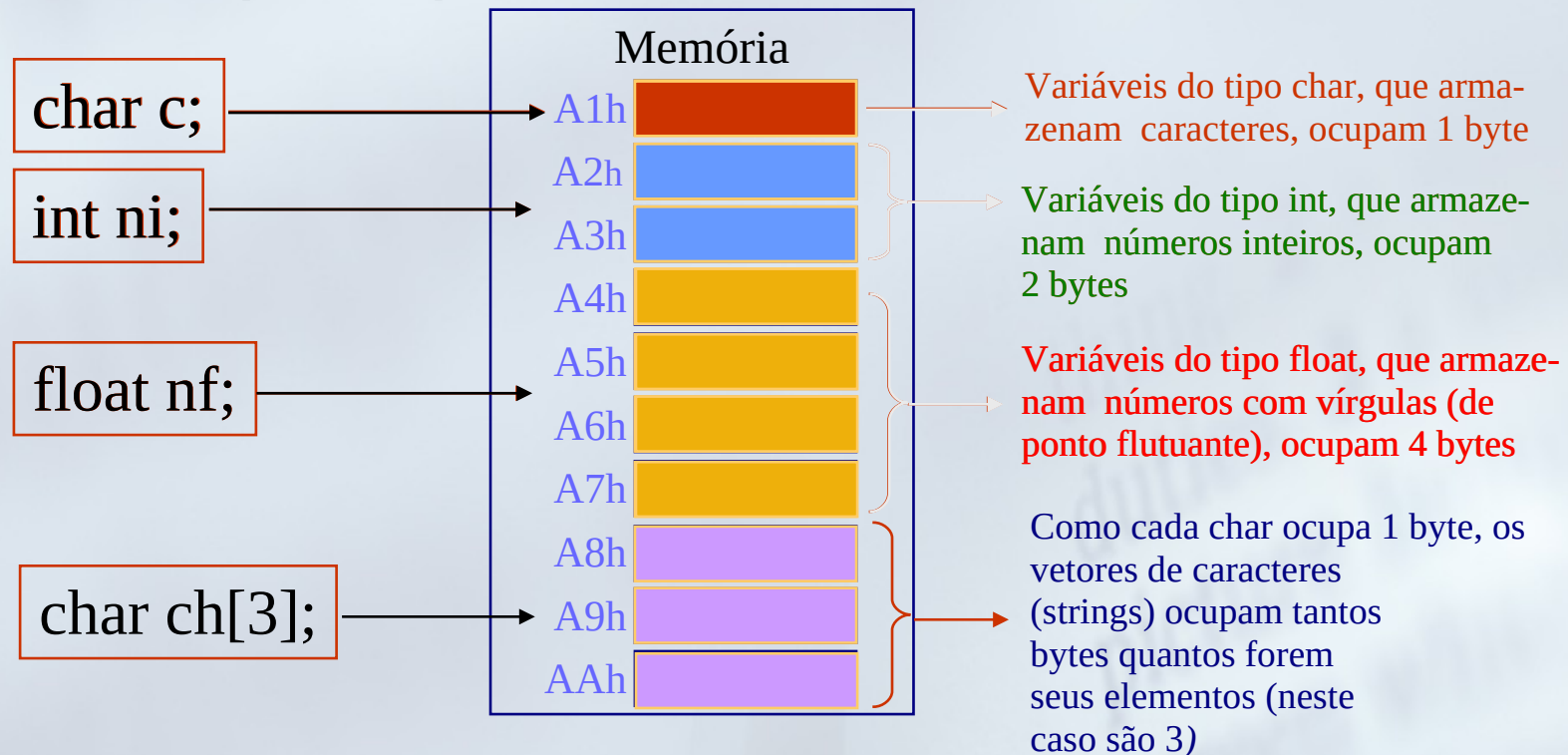
- Struct

Aula de Hoje!

- Ponteiros.

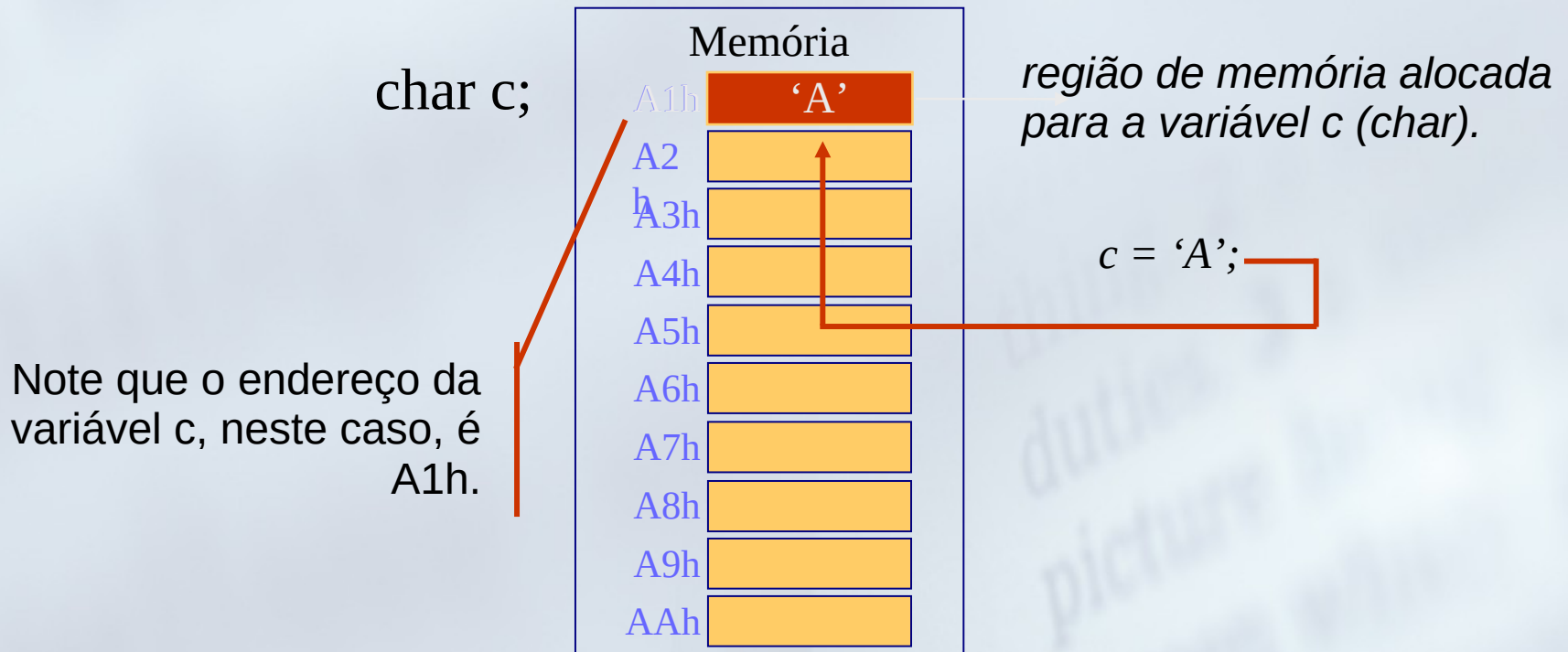
Variáveis *versus* Memória

- Quando você declara uma variável, o compilador reserva uma região de memória para ela.
- Essa região de memória é reservada de acordo com o tamanho do tipo de dado para o qual a variável foi declarada.



Variáveis *versus* Memória

- ◆ Portanto, cada variável fica em um endereço de memória diferente;
- ◆ Quando fazemos uma atribuição a uma variável, o valor atribuído é colocado na região de memória que foi reservada para ela.

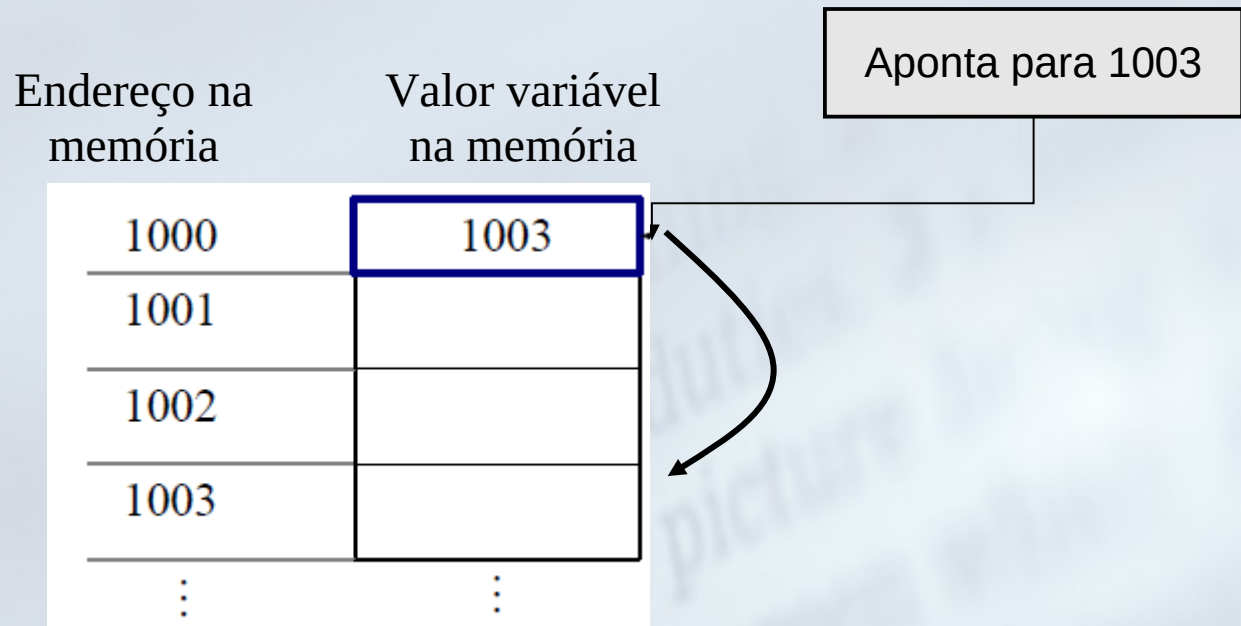


Ponteiros

- Funções modificam parâmetros através de ponteiros
→ **Passagem por referência.**
- Suportam alocação dinâmica de memória.

O que é um ponteiro?

- É uma variável que contém um endereço de memória.
- Usualmente esse é o endereço de outra variável na memória



Declarando Ponteiros

- Os ponteiros são iguais às variáveis comuns. A única diferença está no fato dele armazenar um endereço, e não um dado ou valor.
- Para informar que você quer declarar um ponteiro, e não uma variável comum, coloque o símbolo de asterisco ao lado do tipo da variável:

char *ponteiro;

- O símbolo de asterisco é que vai indicar ao C que você quer um ponteiro e não uma variável comum. Fora esse detalhe, a declaração é idêntica a de uma variável comum.

Declarando Ponteiros

- Para cada tipo de variável há um ponteiro específico.
- Isso significa que se você vai armazenar o endereço de uma variável do tipo `int`, deve criar um ponteiro para `int` (que é `int*`). Se for `char`, um ponteiro para `char` (`char*`).

```
char *pc;
```

- No exemplo acima, o C vai saber que você está criando uma variável chamada `pc` que vai armazenar endereços de variáveis do tipo `char`.

```
int *pn;
```

- No exemplo acima, o C vai saber que você está criando uma variável chamada `pn` que vai armazenar endereços de variáveis do tipo `int`.

Variáveis ponteiro

- Declaração:

tipo *variavel;

* indica que é um
ponteiro

- Exemplos

char	*ch;
int	*i;
float	*num;

Declaração de Variável x Ponteiro

- **Alguns exemplos:**

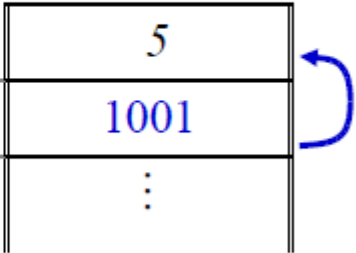
Tipo	Variável comum	Ponteiro p/o tipo
char	char letra;	char* pontLetra;
int	int numero;	int* pontNumero;
float	float num;	float* pontNum;

Operador &

- Devolve o endereço de memória de seu operando

```
int count;  
int *m;  
  
count = 5;  
  
m = &count;
```

Variável	Endereço	Memória
count	1001	5
m	1200	1001
		⋮



Operador *

- Devolve o valor da variável localizada no endereço de seu operando

```
int count, q;  
int *m;
```

```
count = 5;  
m = &count;
```

```
q = *m;
```

Variável	Endereço	Memória
count	1001	5
m	1200	1001
q	1007	5
		⋮

Operador *

- Devolve o valor da variável localizada no endereço de seu operando

```
int count, q;  
int *m;
```

```
count = 5;  
m = &count;  
q = *m;
```

```
*m = 10;
```

Variável	Endereço	Memória
count	1001	?
m	1200	?
q	1007	?
		⋮

Valor de x ?

```
#include <stdio.h>

int main(void){

    int x;
    int *i;

    x = 23;
    i = &x;
    *i = 19;

    printf("x = %d",x);

    return 0;
}
```

Valor de i ?

```
#include <stdio.h>

int main(void){

    int x;
    int *i;

    x = 23;
    i = &x;
    *i = x;

    printf("i = %d",*i);

    return 0;
}
```


Atribuição =

- Atribui a referência ao ponteiro.

```
int count = 5;  
int *m;  
int *n;  
  
m = &count;  
n = m;
```

Variável	Endereço	Memória
count	1001	5
m	1200	1001
n	1300	1001

Atribuição de ponteiros

```
#include <stdio.h>

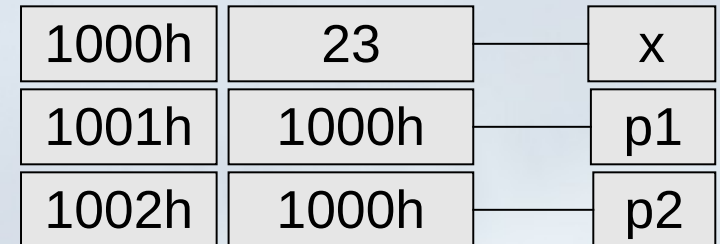
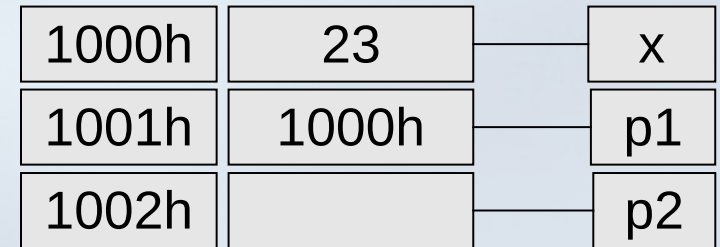
int main(void){

    int x;
    int *p1, *p2;
    x = 23;
    p1 = &x;

    p2 = p1;

    printf("p2:  %p \n", p2);
    printf("p1:  %p \n", p1);
    printf("&x:  %p \n", &x);

    return 0;
}
```



```
p2: 1000h
p1: 1000h
&x: 1000h
```

Atribuição de ponteiros

```
#include <stdio.h>

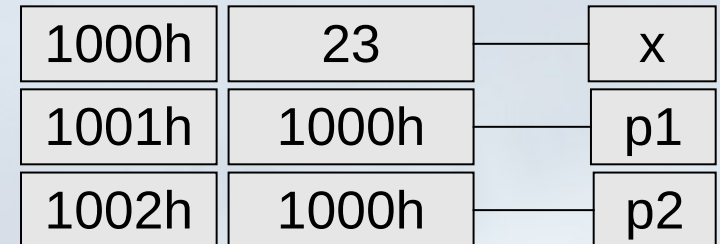
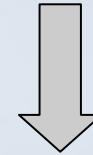
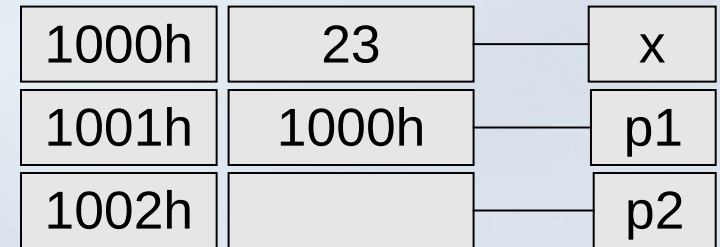
int main(void){

    int x;
    int *p1, *p2;
    x = 23;
    p1 = &x;

    p2 = p1;

    printf("*p2: %d \n", *p2);
    printf("*p1: %d \n", *p1);
    printf("  x: %d \n", x);

    return 0;
}
```



```
*p2: 23
*p1: 23
x: 23
```

Atribuição de ponteiros

```
#include <stdio.h>

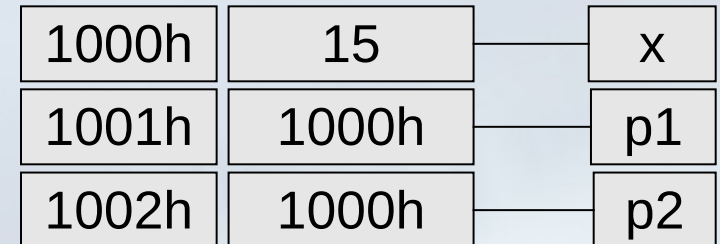
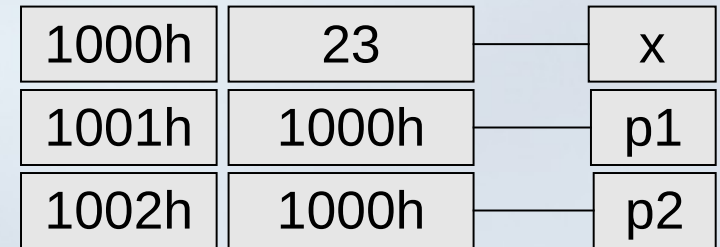
int main(void){
    int x;
    int *p1, *p2;

    x = 23;
    p1 = &x;

    p2 = p1;
    x = 15;

    printf("*p2: %d \n", *p2);
    printf("*p1: %d \n", *p1);
    printf("&x: %p \n", &x);

    return 0;
}
```



```
*p2: 15
*p1: 15
&x: 1000h
```

Utilizando Ponteiros

- Utilizar um ponteiro é simples. Como qualquer variável, para colocar um valor dentro dela, basta atribuí-lo:

- `char* ponteiro = 10030; //coloca o endereço 10030 no ponteiro`

- Porém, os ponteiros são mais utilizados para armazenar o endereço de outras variáveis:

- `char c = 'A'; // coloca o valor 'A' na variável c`

- `char* ponteiro = &c; // coloca o endereço da variável c no ponteiro`

- Note que o operador `&` (que retorna o endereço de uma variável) foi utilizado!

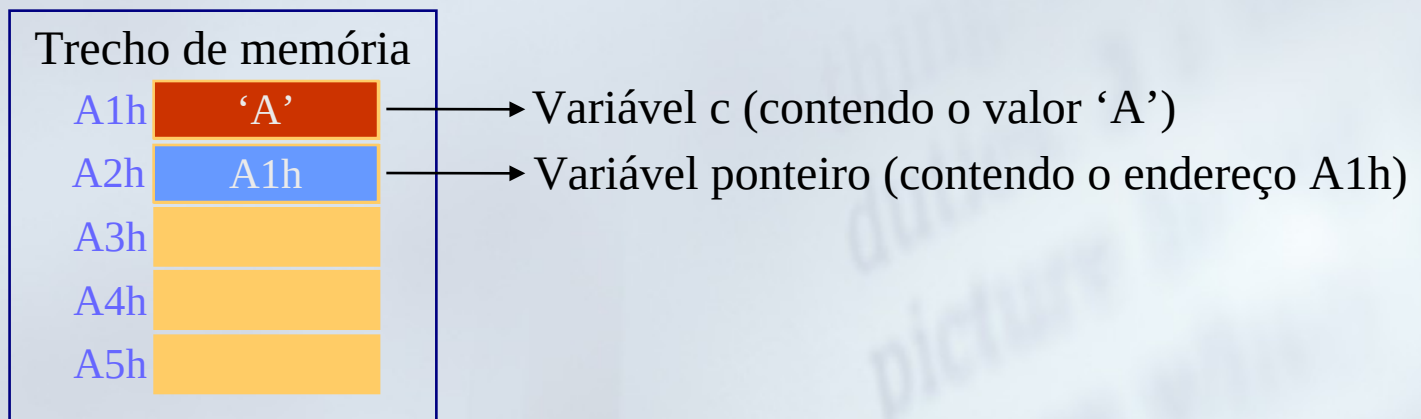
- **OBS: A princípio, você não pode atribuir endereços de variáveis que não sejam do mesmo tipo do ponteiro:** `int a = 20;`

- `char* pont = &a; // Errado, pois a não é do tipo char!`

Utilizando Ponteiros

- Se você mandar imprimir uma variável do tipo ponteiro, **ela vai mostrar o endereço armazenado nela:**

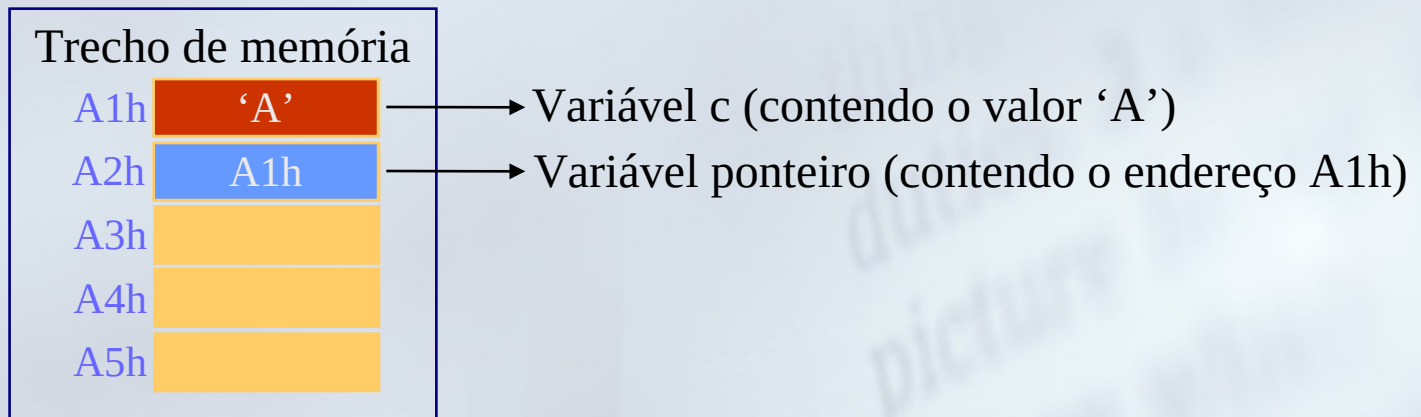
```
char c = 'A';           // coloca o valor 'A' na variável c
char *ponteiro = &c;    // coloca o endereço da variável c no ponteiro
printf("%p", ponteiro); // imprime o conteúdo do ponteiro, que é o
                        // endereço da variável c
```



Utilizando Ponteiros

- ◆ Você pode utilizar o ponteiro para mostrar ou manipular o conteúdo do endereço (da variável) que ele aponta.
- ◆ Para fazer isso, você deve utilizar o operador `*` :

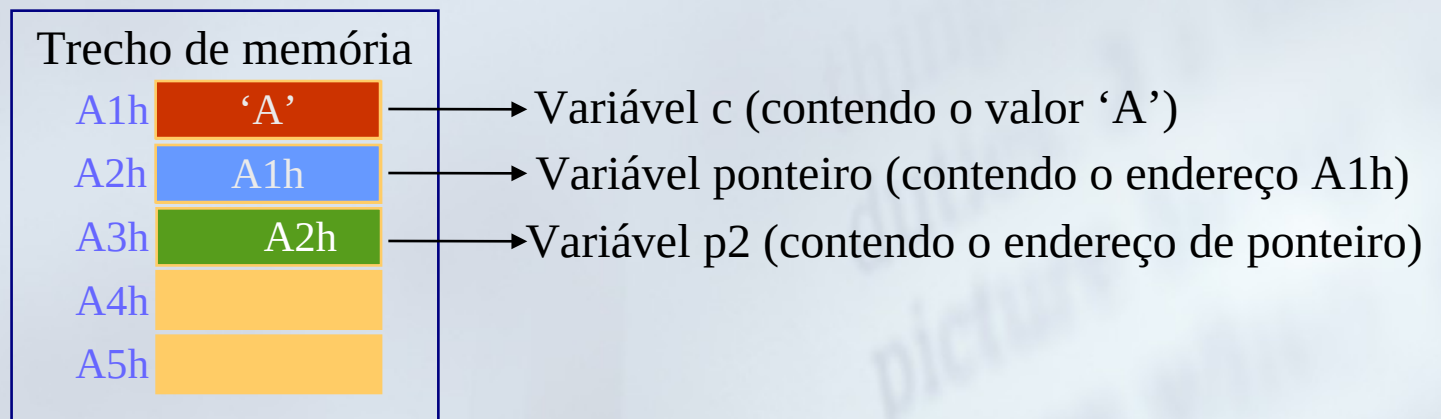
```
char c = 'A';           // coloca o valor 'A' na variável c
char *ponteiro = &c;    // coloca o endereço da variável c no ponteiro
printf("%c", *ponteiro); // imprime o conteúdo do endereço para o
                        // qual o ponteiro aponta ( que é 'A' )
```



Utilizando Ponteiros

- ◆ Ponteiros também têm endereço. Logo, você também pode imprimir o seu endereço ou armazená-lo ou utilizá-lo em um outro ponteiro:

```
char c = 'A';           // coloca o valor 'A' na variável c
char *ponteiro = &c;    // coloca o endereço da variável c no ponteiro
printf("%p", &ponteiro); // imprime o endereço do ponteiro (A2h)
char **p2 = &ponteiro;  // cria um ponteiro de ponteiro, e coloca o
// endereço do ponteiro lá
```



Exercício 1

```
int main(){
    int x;
    int *p1, *p2;
    x = 100;
    p1 = &x;

    p2 = p1;
    x = 50;

    printf("p2:  %p \n", p2);
    printf("p1:  %p \n", p1);
    printf("&x:  %p \n", &x);

    return 0;
}
```

//Neste caso?

```
printf("p2:  %d \n", *p2);
printf("p1:  %d \n", *p1);
printf("x:    %d \n", x);
```

O que será impresso
na tela?

Exercício 1

```
int main(){
    int x;
    int *p1, *p2;
    x = 100;
    p1 = &x;

    p2 = p1;
    x = 50;
```

```
    printf("p2:  %p \n", p2);
    printf("p1:  %p \n", p1);
    printf("&x:  %p \n", &x);
```

```
    return
}
```

```
p2:  50
p1:  50
x:  50
Pressione qualquer tecla para continuar. . . _
```

//Neste caso?

```
    printf("p2:  %d \n", *p2);
    printf("p1:  %d \n", *p1);
    printf("&x:  %d \n", x);
```

```
p2:  0028FF44
p1:  0028FF44
&x:  0028FF44
Pressione qualquer tecla para continuar. . . _
```

Exercícios

```
int main(){
    int *p3;
    int **p4;
    int y = 100;
    p3 = &y;
    p4 = &p3;
    printf("&p3: %p \n",&p3);
    printf("p3: %p \n",p3);
    printf("&p4: %p \n",&p4);
    printf("p4: %p \n",p4);
    printf("&y: %p \n",&y);
    printf("y: %d \n",y);
    printf("*p3: %d \n",*p3);
    printf("*p4: %p \n",*p4);
    printf("**p4: %d \n",**p4);
}
```

Exercícios

```
int main(){
    int *p3;
    int **p4;
    int y = 100;
    p3 = &y;
    p4 = &p3;
    printf("&p3: %p \n",&p3);
    printf("p3: %p \n",p3);
    printf("&p4: %p \n",&p4);
    printf("p4: %p \n",p4);
    printf("&y: %p \n",&y);
    printf("y: %d \n",y);
    printf("*p3: %d \n",*p3);
    printf("*p4: %p \n",*p4);
    printf("**p4: %d \n",**p4);
}
```

C:\Users\Diego\Desktop\UTFPR\Disciplinas - 02-2012\LT31A-

```
&p3: 0028FF44
p3: 0028FF3C
&p4: 0028FF40
p4: 0028FF44
&y: 0028FF3C
y: 100
*p3: 100
*p4: 0028FF3C
**p4: 100
```

Pressione qualquer tecla para continuar. . .

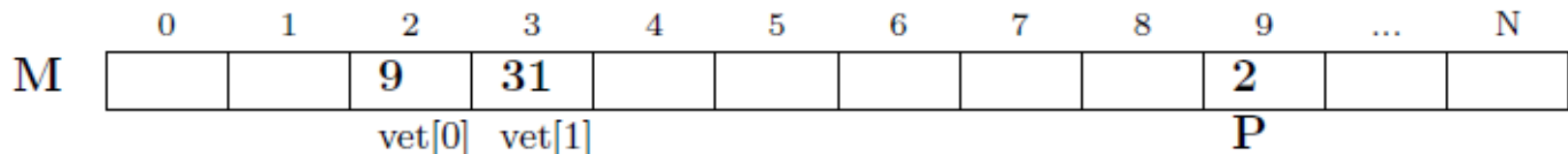
Aritmética de ponteiros

- É interessante observar que, embora ponteiros sejam considerados um tipo especial de dados, eles ainda são números e, como tal, faz sentido realizar algumas operações matemáticas sobre eles.
- A aritmética de ponteiros **é restrita apenas a soma, subtração e comparações.**

```
int vet[2] = {9, 31};
```

```
int *P;
```

```
p = &vet[0];  $\Rightarrow$  faz P apontar para a 1ª célula de vet
```



Aritmética de ponteiros

Nesse caso, o conteúdo de $*P$ é 9. Se for realizada uma adição sobre P :

$P++$;

Teremos:

	0	1	2	3	4	5	6	7	8	9	...	N
M			9	31						3		
			vet[0]	vet[1]						P		

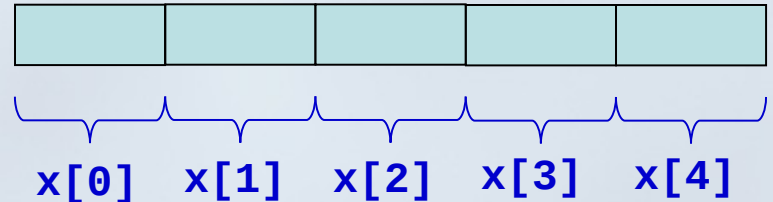
Agora o conteúdo de $*P$ é **31**

Vetores em C

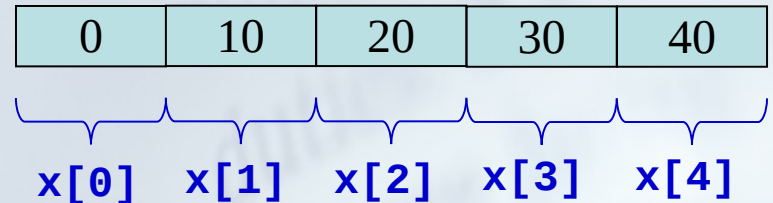
```
int x[5];
```



Posições contíguas de memória



```
int i;  
for (i= 0; i < 5; i++){  
    x[i]= i*10;  
}
```



Vetor é um ponteiro em C

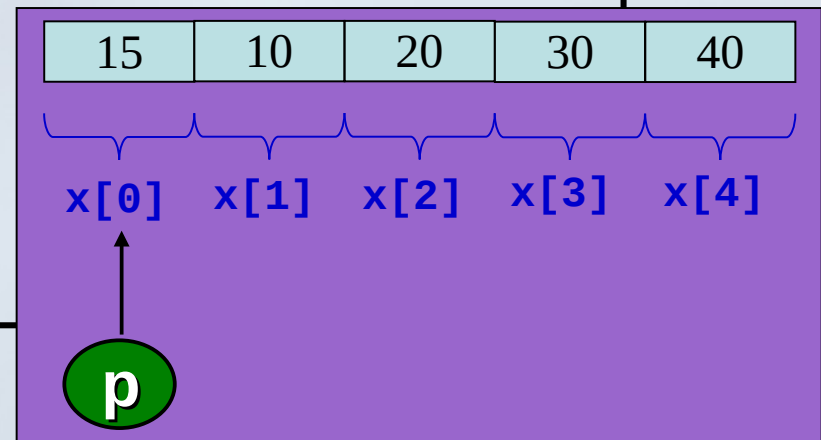
```
int x[5];  
int *p;
```

```
p = &x[0]; // p aponta para a primeira posição de x
```

```
x[0] = 15; // *p é igual a 15;
```

```
printf("x[0]: %d \n", x[0]);  
printf("  *p: %d \n", *p);
```

- **x[0]** é a primeira posição do vetor. Qualquer alteração feita a ele, altera primeiro elemento do vetor.
- **p** aponta para a primeira posição do vetor, consequentemente qualquer alteração feita a ele altera a primeira posição do vetor também.



```
x[0]: 15  
*p: 15
```


Vetor é um ponteiro em C

```
#include <stdio.h>

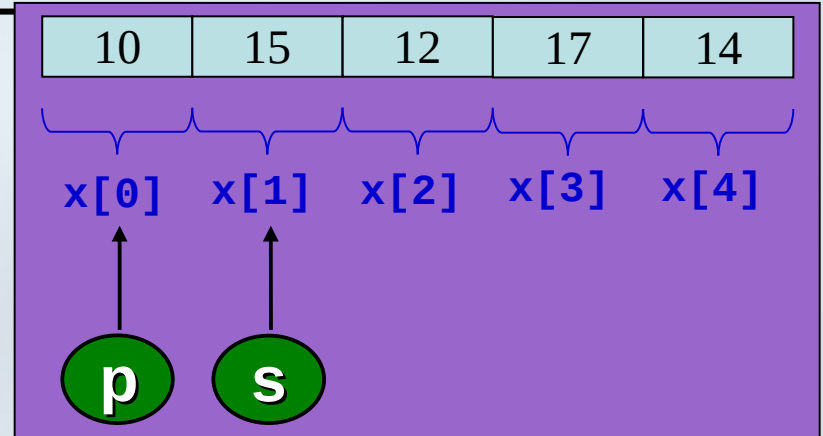
int main(void){

    int x[5]={10,15,12,17,14};
    int *p,*s;

    p = x;
    s = p + 1;

    printf("  p:  %d \n", *p );
    printf("  s:  %d \n", *s );

    return 0;
}
```



p: 10
s: 15

Exercícios

1. Dado um vetor de inteiros, escreva este utilizando ponteiros:

```
1 #include <stdio.h>
2
3 int main(void) {
4
5     int i ;
6     int *p ;
7     int vetor[] = {10, 20, 30, 40, 50};
8
9     for (p = vetor; p < &vetor[5]; p++)
10     {
11         printf("Endereco [%p] - Valor: %d \n", p, *p);
12     }
13
14
15     system("PAUSE");
16 }
```

Exercícios

Dada uma palavra qualquer escreva a primeira letra desta palavra, a segunda, a sexta e a oitava:

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main ()
5 {
6     char str[ ] = "Eng Eletronica";
7     char *p;
8     p = str;
9
10    printf ("*p = %c\n", *p);
11    printf ("*(p+1) = %c\n", *(p+1));
12    printf ("*(p+5) = %c\n", *(p+5));
13    printf ("*(p+7) = %c\n", *(p+7));
14
15    system("PAUSE");
16
17 }
```

```
*p = E
*(p+1) = n
*(p+5) = l
*(p+7) = t
Pressione qualquer tecla para continuar. . .
```

Funções

- Chamada por valor
- Chamada por referência

Chamada por valor

```
#include <stdio.h>

int soma(int a, int b);

int soma(int a, int b){
    int res;
    res= a + b;
    return res;
}

int main(void){
    int val1, val2, resp;
    val1= 10;
    val2= 15;
    resp = soma(val1, val2);
    printf("soma= %d", resp);

    return 0;
}
```

Chamada por referência

```
void troca(int *a, int *b);

void troca(int *a, int *b){
    int temp;
    temp= *a;
    *a= *b;
    *b= temp;
}

int main(void){
    int val1, val2;
    val1= 10;
    val2= 15;
    printf("val1= %d      val2= %d \n", val1, val2);
    troca(&val1,&val2);
    printf("val1= %d      val2= %d \n", val1, val2);

    return 0;
}
```

Chamada por referência -

•Arranjos (vetores, matrizes, ...) são sempre passados por referência.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void geraMat(int mat[3][3])
5 {
6     int i, j;
7     for(i= 0; i < 3; i++)
8     {
9         for (j= 0; j < 3; j++)
10         {
11             printf("Digite um valor para [%d][%d]", i,j);
12             scanf("%d", &mat[i][j]);
13         }
14     }
15 }
16
17 int main()
18 {
19     int mat[3][3];
20     int i, j;
21
22     geraMat(mat);
23     for(i= 0; i < 3; i++)
24     {
25         for (j= 0; j < 3; j++)
26         {
27             printf("[%d] ", mat[i][j]);
28         }
29         printf("\n");
30     }
31     system("PAUSE");
32 }
```

Aula de Hoje

- Ponteiros e seus operadores.
- Expressão com ponteiros (atribuição, aritmética e comparação)
- Ponteiros e arranjos.

Exercícios

Exercício 01.: Escreva um programa que contenha duas variáveis inteiras. Compare seus endereços e exiba o maior endereço.

■ Exercício: Faça uma função que receba por parâmetro dois ponteiros para inteiro - "void troca(int* a, int *b)", e em seguida troque o seus valores.

Exercícios

Exercício 02.: Considere a seguinte declaração: `int a,*b,**c,***d;` Escreva um programa que leia a variável `a` e calcule e exiba o dobro, o triplo e o quádruplo desse valor utilizando apenas os ponteiros `b`, `c` e `d`. O ponteiro `b` deve ser usada para calcular o dobro, `c` o triplo e `d` o quádruplo.

Exercícios

Exercício 02.: Escreva uma função que dado um número real passado como parâmetro, retorne a parte inteira e a parte fracionaria deste número. Escreva um programa que chama esta função.
Assinatura: `void frac(float num, int* inteiro, float* frac);`

Dúvidas, Críticas ou Sugestões

diegobertolini@utfpr.edu.br